ABSTRACT

This assignment designs and evaluates uninformed search strategies for an autonomous hospital medicine-delivery robot. The hospital is represented as a graph in which locations are states and corridors are actions with positive travel-time costs. Breadth-First Search (BFS), Uniform-Cost Search (UCS), and Iterative Deepening Depth-First Search (IDDFS) are implemented and compared. Their basic operations, time and space requirements, recurrence relations, and best-, average-, and worst-case behaviour are analysed. A reproducible binary-tree test graph is used to collect node-generation, node-expansion, frontier-size, path-depth, path-cost, and execution-time measurements at four state-space sizes. Finally, the design is reconsidered when localization becomes uncertain and the robot must reason over a set of possible locations (a belief state).

INTRODUCTION

Artificial Intelligence enables an autonomous agent to perceive its environment, choose actions, and achieve goals. In a hospital, a medicine-delivery robot must navigate corridors and junctions safely while minimizing travel time. Search algorithms provide a systematic way to find a route from the pharmacy to a ward. As the hospital expands, the number of possible locations increases, making memory and processing efficiency important. This assignment compares three uninformed strategies and evaluates how uncertain localization changes the original problem.

TASK 1 – DESIGN THE AGENT AND SEARCH ALGORITHMS

PEAS description

Table 1. PEAS description of the hospital robot

PEAS element	Description
Performance measure	Deliver the correct medicine to the requested ward; minimize travel time, path length, collisions, and missed deliveries; obey safety rules.
Environment	Hospital corridors, junctions, lifts, pharmacy, wards, people, doors, and obstacles.
Actuators	Drive motors, steering, braking, lift interface, door interface, display/alert system.
Sensors	Wheel encoders, camera, LiDAR/ultrasonic sensors, RFID/QR markers, lift/door status sensors, localization system.

Environment classification

Table 2. Environment classification

Property	Classification	Justification
Observability	Partially observable	People, doors, and sensor noise can hide or distort the robot's exact surroundings.
Determinism	Mostly deterministic	A commanded move normally leads to the connected location, but people and faults can introduce uncertainty.
Episodes	Sequential	Each move affects the robot's later position and available route.
Dynamics	Dynamic	People, carts, doors, and lift availability can change while the robot plans.
State/action space	Discrete planning model	Junctions and corridors are represented as discrete graph nodes and edges; physical motion is continuous.
Agents	Multi-agent environment	Patients, staff, visitors, and other robots can affect movement.

AI view and rationality

The robot follows the acting-rationally view of AI. It selects actions expected to achieve the delivery goal while minimizing travel cost and respecting safety constraints. It is a rational problem-solving agent because it uses a goal, a model of available actions, and a performance measure to choose a route. This is connected to the early history of AI, when researchers studied symbolic problem solving through explicit state spaces, operators, and search procedures (for example, maze solving and theorem proving).

Problem formulation

Table 3. Search problem formulation

Component	Definition
State	Current hospital location (node ID).
Initial state	Central pharmacy node, represented by 0 in the test graph.
Actions	Move along an available outgoing corridor to an adjacent node.
Transition model	Result (u, move-to-v) = v when edge (u,v) exists.
Goal test	Current node equals the requested ward node.
Path cost	Sum of positive corridor travel-time costs along the route.

SELECTED UNINFORMED STRATEGIES

A. Breadth-First Search (iterative queue)

BFS explores the shallowest unexpanded node first using a FIFO queue. It is complete on a finite graph when duplicate states are detected. It returns a minimum-edge route when every edge has equal cost, but it does not generally minimize travel time when edge costs differ. It is selected as a strong baseline for shallow routes.

Pseudocode: put start in queue; mark start visited; while queue is not empty, remove front node; if it is goal, reconstruct path; otherwise expand its unvisited neighbours, record parents, and append them.

B. Uniform-Cost Search (iterative priority queue)

UCS expands the frontier node with the smallest accumulated path cost g(n), using a min-priority queue. With strictly positive step costs, it is complete and returns a least-cost route. It is selected because hospital corridors have different travel times, making cost optimality important.

Pseudocode: insert start with cost 0; repeatedly remove the lowest-cost entry; ignore stale entries; if it is goal, return its path; otherwise relax each outgoing edge and insert any improved path cost.

C. Iterative Deepening Depth-First Search (recursive depth-limited search)

IDDFS runs a recursive depth-limited DFS repeatedly with limits 0, 1, 2, and so on. It combines the shallow-solution behaviour of BFS with the low memory use of DFS. It is selected to investigate a recursive strategy that is practical when the solution depth is small compared with the total number of states. It is optimal only when step costs are equal.

Pseudocode: for limit = 0,1,2,..., run recursive DLS (start,limit); DLS returns success if node is goal, failure if limit is zero, otherwise recursively explores each successor with limit-1.

2.6 Preprocessing

The experiment uses a compact implicit binary-tree graph: node IDs are 0 to N−1 and children of u are 2u+1 and 2u+2 when below N. The pharmacy is node 0 and the test ward is node N−1. Edge costs are positive and computed deterministically from the child ID and seed. This representation avoids allocating a million separate adjacency lists. Graph description and start/goal setup require O(1) extra preprocessing; generating an explicit adjacency list instead would require Θ(V+E) time and space. Search bookkeeping (visited/parent maps, best-cost map, queue or heap) is counted separately.

TASK 2 – BASIC OPERATION

Table 4. Basic operations

Algorithm	Basic operation	Where and why it dominates
BFS	Successor examination / enqueue of a newly discovered node	Performed when a node is expanded. It determines how many states are visited and how quickly the FIFO frontier grows.
UCS	Priority-queue insertion and delete-min (heap operations), plus edge relaxation	Each expanded edge may trigger a heap update; heap work grows logarithmically with frontier size and reflects cost-based ordering.
IDDFS	Recursive node visit and successor examination	Repeated at each depth limit. Shallow levels are revisited, so total calls determine its overhead.

TASK 3 – MATHEMATICAL TIME AND SPACE COMPLEXITY

Let b be the branching factor, d the depth of the shallowest goal, m the maximum depth, C* the optimal solution cost, and ε the minimum positive step cost. The number of nodes at depth i in a full b-ary tree is b^i.

4.1 BFS derivation

In the worst case BFS generates levels 0 through d: $1 + b + b^2 + ... + b^d$. By the geometric-series identity, this is $(b^{(d+1)}-1)/(b-1) = \Theta(b^d)$ for $b>1$. Each generated node is inserted and removed at most once, so time is $\Theta(b^d)$. The largest frontier is the last generated level, approximately b^d, and the parent/visited structure stores discovered nodes; therefore space is $\Theta(b^d)$. With explicit adjacency-list preprocessing, add $\Theta(V+E)$; with the compact implicit test representation, graph setup is O(1).

4.2 UCS derivation

UCS expands nodes in nondecreasing path cost. With minimum step cost ε, any path of cost at most C* has depth no greater than floor(C*/ε). The number of nodes within that depth bound is at most $1 + b + ...$

+ b^floor(C*/ε) = Θ(b^(C*/ε)) in the general worst-case bound. Each edge relaxation may cause a heap insertion/update costing O(log F), where F is frontier size. Thus a useful bound is O((V_r+E_r) log V_r) for the explored portion of a finite graph; in a tree-style bound this is O(b^(1+C*/ε) log(b^(C*/ε))). Heap and best-cost/parent records require O(V_r) space. UCS is cost optimal for positive step costs.

4.3 IDDFS derivation

At limit d, depth-limited DFS may visit 1+b+...+b^d nodes. Across limits 0 to d, nodes at level i are revisited d−i+1 times. Total visits are Σ (i=0 to d) [(d−i+1)b^i]. The deepest level dominates for b>1, giving Θ(b^d) time (with a constant-factor re-expansion overhead relative to BFS). The recursion path stores at most d+1 nodes, so auxiliary search space is Θ(bd) when successor storage is included, commonly stated as O(bd), or O(d) for a tree with generated successors one at a time.

4.4 Preprocessing and overall cost

For an explicit adjacency list, preprocessing is Θ(V+E) time and space. Overall cost is preprocessing plus search: BFS Θ(V+E)+Θ(b^d); UCS Θ(V+E)+heap-based explored-edge cost; IDDFS Θ(V+E)+Θ(b^d). The experiment's implicit graph has O(1) graph-description setup and computes successors on demand, so the measured search does not include a separately materialized adjacency list.

TASK 4 – BEST, AVERAGE AND WORST CASE

Table 5. Case analysis

Algorithm	Best case	Average case	Worst case
BFS	Goal is start or first shallow successor; very few nodes expanded.	Goal is distributed among frontier nodes; about half the nodes at the goal depth may be examined under a simple uniform-position assumption.	Goal is deepest or unreachable; all nodes through depth d (or all reachable states) are generated.
UCS	Goal is removed from the priority queue before many alternatives because its cost is very low.	Depends on edge-cost distribution; all states with path cost below C* and some equal-cost ties are expanded.	Many low-cost detours precede the goal, or goal is unreachable; a large fraction of reachable states is expanded.
IDDFS	Goal at depth 0 or 1; only a few recursive calls.	Goal at depth d and not first in successor order; shallow levels are repeatedly revisited.	Goal is last at maximum depth or unreachable; all depth-limited searches are exhausted.

For a full b-ary tree, BFS worst-case generated nodes through depth d sum to (b^(d+1)−1)/(b−1). IDDFS revisits upper levels: its exact count is Σ(d−i+1) b^i. UCS depends on cumulative path costs rather than depth, so cheap detours can make it expand many more nodes than the final route alone. The test graph is a finite tree; the unreachable test uses a goal ID outside the graph.

TASK 5 – ASYMPTOTIC ANALYSIS

Table 6. Asymptotic comparison

Algorithm	Time growth	Space growth	Meaning for hospital
BFS	Θ(b^d) tree worst case	Θ(b^d)	Fast for shallow routes, but queue and parent map can exhaust memory as branching/depth grow.
UCS	O((V_r+E_r) log V_r) on finite explored graph	O(V_r)	Appropriate when travel times differ; heap memory and ordering overhead are the trade-off.
IDDFS	Θ(b^d)	O(bd)	Much lower memory than BFS, but repeated expansions add work.

When the hospital grows from 1,000 to 1,000,000+ locations, the number of states alone does not determine difficulty: branching factor, route depth, and edge costs also matter. BFS can become memory-limited first because its frontier may contain an entire broad level. IDDFS uses much less memory but may spend more time repeating expansions. UCS is preferable when travel-time optimality matters, provided its priority queue and explored-state map fit available memory.

TASK 6 – RECURRENCE RELATION

For recursive depth-limited search on a regular b-ary tree, let T(l) be the maximum work with remaining depth limit l. At l=0, the algorithm checks the goal and returns, so T(0)=c. For l>0, it performs constant local work c_1 and may recursively inspect up to b children, each with limit l−1: T(l)=bT(l−1)+c_1. Here the original problem size is l, each subproblem has size l−1, and there are at most b recursive calls.

TASK 7 – SOLVE THE RECURRENCE

Use repeated substitution. T(l)=bT(l−1)+c_1. Substituting gives T(l)=b[bT(l−2)+c_1]+c_1=b^2T(l−2)+c_1(b+1). After j substitutions, T(l)=b^jT(l−j)+c_1Σ(i=0 to j−1)b^i. Set j=l: T(l)=b^lT(0)+c_1(b^l−1)/(b−1). For b>1, T(l)=Θ(b^l). The recursion tree has b^i calls at level i, so the deepest level dominates. IDDFS repeats this search for limits 0 through d; total calls are Σ(i=0 to d)(d−i+1)b^i=Θ(b^d), while memory remains O(bd).

TASK 8 – IMPLEMENT AND TEST THE ALGORITHMS

The following Python 3 implementation is the complete reproducible program used for the experiment. It uses a compact implicit graph, positive edge costs, and the three algorithms analysed above. The same start and goal are used for each algorithm at a given N.

Python Program:

```
import heapq
import math
class HospitalGraph:
    def __init__(self, n):
        self.n = n
    def neighbours(self, node):
        result = []
        left = 2 * node + 1
        right = 2 * node + 2
        if left < self.n:
            result.append(left)
        if right < self.n:
            result.append(right)
        return result
    def cost(self, node):
        return 1 + ((node * 7) % 9)
def get_path(parent, goal):
    path = []
    while goal is not None:
        path.append(goal)
        goal = parent[goal]
    return path[::-1]
def bfs(graph, start, goal):
    if start < 0 or start >= graph.n:
        return None
    if goal < 0 or goal >= graph.n:
        return None
    queue = [start]
    visited = {start}
    parent = {start: None}
    while queue:
        node = queue.pop(0)
        if node == goal:
            return get_path(parent, goal)
        for neighbour in graph.neighbours(node):
            if neighbour not in visited:
                visited.add(neighbour)
                parent[neighbour] = node
                queue.append(neighbour)
    return None
def ucs(graph, start, goal):
    if start < 0 or start >= graph.n:
        return None
    if goal < 0 or goal >= graph.n:
        return None
    priority_queue = [(0, start)]
    parent = {start: None}
    cost_so_far = {start: 0}
    while priority_queue:
        current_cost, node = heapq.heappop(priority_queue)
        if current_cost != cost_so_far[node]:
            continue
        if node == goal:
            return get_path(parent, goal)
        for neighbour in graph.neighbours(node):
            new_cost = current_cost + graph.cost(node)
            if (neighbour not in cost_so_far or
                    new_cost < cost_so_far[neighbour]):
                cost_so_far[neighbour] = new_cost
                parent[neighbour] = node
                heapq.heappush(
                    priority_queue,
                    (new_cost, neighbour)
                )
    return None
def dls(graph, node, goal, limit, path):
    if node == goal:
        return path
    if limit == 0:
        return None
    for neighbour in graph.neighbours(node):
        if neighbour not in path:
            result = dls(
                graph,
                neighbour,
                goal,
                limit - 1,
                path + [neighbour]
            )
            if result is not None:
                return result
    return None
def iddfs(graph, start, goal):
    if start < 0 or start >= graph.n:
        return None
    if goal < 0 or goal >= graph.n:
        return None
    max_depth = math.floor(math.log2(graph.n))
    for depth in range(max_depth + 1):
        result = dls(
            graph,
            start,
```

```
            goal,
            depth,
            [start]
        )
        if result is not None:
            return result
    return None
def path_cost(graph, path):
    if path is None:
        return None
    total = 0
    for i in range(len(path) - 1):
        total += graph.cost(path[i])
    return total
def test_search(graph, start, goal):
    print("\nStart Node:", start)
    print("Goal Node:", goal)
    algorithms = [
        ("BFS", bfs),
        ("UCS", ucs),
        ("IDDFS", iddfs)
    ]
    for name, algorithm in algorithms:
        result = algorithm(graph, start, goal)
        print("\nAlgorithm:", name)
        if result is None:
            print("Path: Not Found")
            print("Path Cost: Not Available")
        else:
            print("Path:", result)
            print("Path Cost:", path_cost(graph, result))
sizes = [1000, 10000, 100000, 1000000]
for size in sizes:
    print("\n================================")
    print("GRAPH SIZE:", size)
    print("================================")
    graph = HospitalGraph(size)
    print("\nTEST 1: REACHABLE GOAL")
    test_search(graph, 0, size - 1)
    print("\nTEST 2: START EQUALS GOAL")
    test_search(graph, 0, 0)
    print("\nTEST 3: UNREACHABLE GOAL")
    test_search(graph, 0, size + 5)
```

Output:

```
================================
GRAPH SIZE: 1000
================================

TEST 1: REACHABLE GOAL

Start Node: 0
Goal Node: 999

Algorithm: BFS
Path: [0, 2, 6, 14, 30, 61, 124, 249, 499, 999]
Path Cost: 46

Algorithm: UCS
Path: [0, 2, 6, 14, 30, 61, 124, 249, 499, 999]
Path Cost: 46

Algorithm: IDDFS
Path: [0, 2, 6, 14, 30, 61, 124, 249, 499, 999]
Path Cost: 46

TEST 2: START EQUALS GOAL

Start Node: 0
Goal Node: 0

Algorithm: BFS
Path: [0]
Path Cost: 0

Algorithm: UCS
Path: [0]
Path Cost: 0

Algorithm: IDDFS
Path: [0]
Path Cost: 0

TEST 3: UNREACHABLE GOAL

Start Node: 0
Goal Node: 1005

Algorithm: BFS
Path: Not Found
Path Cost: Not Available
```

Test cases and expected behaviour

Table 7. Test cases

Test	Input	Expected behaviour
Reachable goal	N=1,000; start=0; goal=999	Each algorithm returns a valid path from 0 to 999.
Unreachable goal	N=10; start=0; goal=15	Each algorithm exhausts reachable nodes and returns None.
Start equals goal	N=10; start=0; goal=0	Each algorithm returns [0], depth 0, with no expansion.

The graph is a directed binary tree, so every reachable node has exactly one route from the root. BFS and IDDFS return the shallowest route; UCS returns the least-cost route. In this particular tree, the route is unique, so all three find the same route when the goal is reachable.

TASK 9 – EXPERIMENTAL PERFORMANCE

Each configuration was timed five times using seeds 11, 22, 33, 44, and 55. The table reports the mean elapsed time and deterministic operation counts from the final run. Node generation and expansion counts, maximum frontier, solution depth, and UCS path cost are included. Runtime is hardware- and interpreter-dependent; operation counts are the primary evidence.

Python Program:

```
import heapq
import math
import time
import csv
from collections import deque
import matplotlib.pyplot as plt
class HospitalGraph:
    def __init__(self, n):
        self.n = n
    def neighbours(self, node):
        result = []
        left = 2 * node + 1
        right = 2 * node + 2
        if left < self.n:
            result.append(left)
        if right < self.n:
            result.append(right)
        return result
    def cost(self, node):
        return 1 + ((node * 7) % 9)
def get_path(parent, goal):
    path = []
    while goal is not None:
        path.append(goal)
        goal = parent[goal]
    return path[::-1]
```

```python
def bfs(graph, start, goal):
    queue = deque([start])
    parent = {start: None}
    generated = 1
    expanded = 0
    max_frontier = 1
    while queue:
        node = queue.popleft()
        if node == goal:
            break
        expanded += 1
        for neighbour in graph.neighbours(node):
            if neighbour not in parent:
                parent[neighbour] = node
                queue.append(neighbour)
                generated += 1
        max_frontier = max(max_frontier, len(queue))
    if goal not in parent:
        path = None
        cost = None
        depth = None
    else:
        path = get_path(parent, goal)
        cost = calculate_cost(graph, path)
        depth = len(path) - 1
    return {
        "path": path,
        "generated": generated,
        "expanded": expanded,
        "frontier": max_frontier,
        "depth": depth,
        "cost": cost
    }
def ucs(graph, start, goal):
    priority_queue = [(0, start)]
    parent = {start: None}
    cost_so_far = {start: 0}
    generated = 1
    expanded = 0
    max_frontier = 1
    while priority_queue:
        current_cost, node = heapq.heappop(priority_queue)
        if current_cost != cost_so_far[node]:
            continue
        if node == goal:
            break
        expanded += 1
        for neighbour in graph.neighbours(node):
            new_cost = current_cost + graph.cost(node)
            if (neighbour not in cost_so_far or
                    new_cost < cost_so_far[neighbour]):
                cost_so_far[neighbour] = new_cost
                parent[neighbour] = node
                heapq.heappush(
                    priority_queue,
                    (new_cost, neighbour)
                )
                generated += 1
        max_frontier = max(
            max_frontier, len(priority_queue)
        )
```

```
    if goal not in parent:
        path = None
        cost = None
        depth = None
    else:
        path = get_path(parent, goal)
        cost = calculate_cost(graph, path)
        depth = len(path) - 1
    return {
        "path": path,
        "generated": generated,
        "expanded": expanded,
        "frontier": max_frontier,
        "depth": depth,
        "cost": cost
    }
def iddfs(graph, start, goal):
    generated = 0
    expanded = 0
    max_frontier = 0
    def depth_limited(node, limit, path):
        nonlocal generated, expanded, max_frontier
        generated += 1
        max_frontier = max(max_frontier, len(path))
        if node == goal:
            return path
        if limit == 0:
            return None
        expanded += 1
        for neighbour in graph.neighbours(node):
            result = depth_limited(
                neighbour,
                limit - 1,
                path + [neighbour]
            )
            if result is not None:
                return result
        return None
    max_depth = math.floor(math.log2(graph.n))
    path = None
    for depth in range(max_depth + 1):
        path = depth_limited(start, depth, [start])
        if path is not None:
            break
```

```
    if path is None:
        cost = None
        solution_depth = None
    else:
        cost = calculate_cost(graph, path)
        solution_depth = len(path) - 1
    return {
        "path": path,
        "generated": generated,
        "expanded": expanded,
        "frontier": max_frontier,
        "depth": solution_depth,
        "cost": cost
    }
```

```
def benchmark(graph, start, goal, algorithm, runs=5):
    total_time = 0
    result = None
    for i in range(runs):
        start_time = time.perf_counter()
        result = algorithm(graph, start, goal)
        end_time = time.perf_counter()
        total_time += end_time - start_time
    average_time = total_time / runs
    result["time"] = average_time
    return result
sizes = [1000, 10000, 100000, 1000000]
algorithms = {
    "BFS": bfs,
    "UCS": ucs,
    "IDDFS": iddfs
}
all_results = []
for size in sizes:
    print("\n====================================")
    print("GRAPH SIZE:", size)
    print("====================================")
    graph = HospitalGraph(size)
    start = 0
    goal = size - 1
    reference = benchmark(
        graph, start, goal, ucs, 5
    )
    optimal_cost = reference["cost"]
    for name, algorithm in algorithms.items():
        result = benchmark(
            graph, start, goal, algorithm, 5
        )
        if result["cost"] is None:
            optimal = "No"
        else:
            optimal = (
                "Yes" if result["cost"] == optimal_cost
                else "No"
            )
        row = {
            "Graph Size": size,
            "Algorithm": name,
            "Average Time (seconds)": round(result["time"], 6),
            "Nodes Generated": result["generated"],
            "Nodes Expanded": result["expanded"],
            "Maximum Frontier": result["frontier"],
            "Solution Depth": result["depth"],
            "Path Cost": result["cost"],
            "Optimal": optimal
        }

        all_results.append(row)
        print("\nAlgorithm:", name)
        print("Average Time:", row["Average Time (seconds)"])
        print("Nodes Generated:", row["Nodes Generated"])
        print("Nodes Expanded:", row["Nodes Expanded"])
        print("Maximum Frontier:", row["Maximum Frontier"])
        print("Solution Depth:", row["Solution Depth"])
        print("Path Cost:", row["Path Cost"])
        print("Optimal:", row["Optimal"])
```

```
filename = "task9_results.csv"
with open(filename, "w", newline="") as file:

    fieldnames = all_results[0].keys()

    writer = csv.DictWriter(
        file, fieldnames=fieldnames
    )
    writer.writeheader()
    writer.writerows(all_results)
print("\nResults saved to", filename)
metrics = [
    ("Average Time (seconds)", "Execution Time"),
    ("Nodes Generated", "Nodes Generated"),
    ("Maximum Frontier", "Maximum Frontier Size")
]
for metric, title in metrics:
    plt.figure()
    for name in algorithms:
        x_values = []
        y_values = []
        for row in all_results:
            if row["Algorithm"] == name:
                x_values.append(row["Graph Size"])
                y_values.append(row[metric])
        plt.plot(
            x_values,
            y_values,
            marker="o",
            label=name
        )
    plt.xscale("log")
    plt.xlabel("Graph Size")
    plt.ylabel(title)
    plt.title(title + " Comparison")
    plt.legend()
    plt.grid(True)
    image_name = title.lower().replace(" ", "_") + ".png"
    plt.savefig(image_name)
    plt.show()
print("\nAll performance graphs generated successfully.")
```

Output:

```
====================================
GRAPH SIZE: 1000
====================================

Algorithm: BFS
Average Time: 0.000461
Nodes Generated: 1000
Nodes Expanded: 999
Maximum Frontier: 500
Solution Depth: 9
Path Cost: 46
Optimal: Yes
```

```
Nodes Generated: 936
Nodes Expanded: 825
Maximum Frontier: 264
Solution Depth: 9
Path Cost: 46
Optimal: Yes

Algorithm: IDDFS
Average Time: 0.000838
Nodes Generated: 1994
Nodes Expanded: 994
Maximum Frontier: 10
Solution Depth: 9
Path Cost: 46
Optimal: Yes

====================================
GRAPH SIZE: 10000
====================================

Algorithm: BFS
Average Time: 0.004542
Nodes Generated: 10000
Nodes Expanded: 9999
Maximum Frontier: 5000
Solution Depth: 13
Path Cost: 65
Optimal: Yes

Algorithm: UCS
Average Time: 0.009676
Nodes Generated: 9412
Nodes Expanded: 8533
Maximum Frontier: 2208
Solution Depth: 13
Path Cost: 65
Optimal: Yes

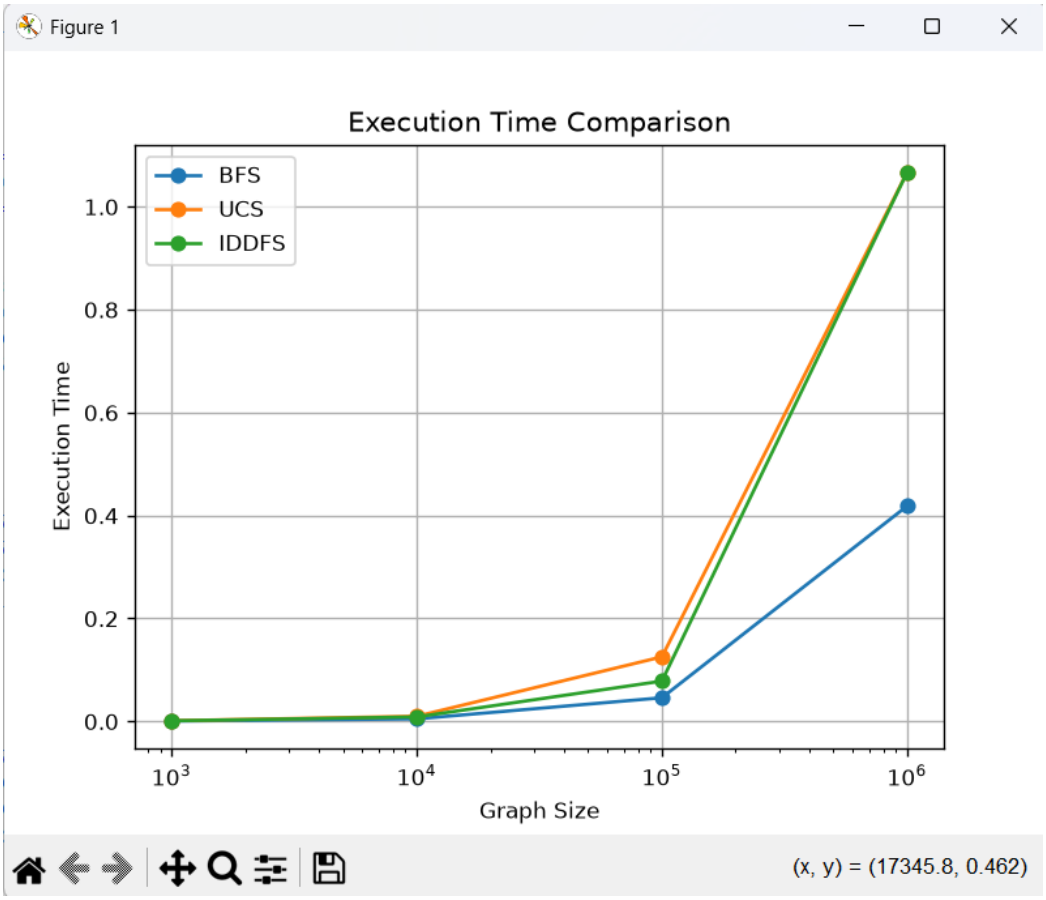
Algorithm: IDDFS
Average Time: 0.008411
Nodes Generated: 19995
Nodes Expanded: 9995
Maximum Frontier: 14
Solution Depth: 13
Path Cost: 65
Optimal: Yes
```

```
====================================
GRAPH SIZE: 100000
====================================

Algorithm: BFS
Average Time: 0.045586
Nodes Generated: 100000
Nodes Expanded: 99999
Maximum Frontier: 50000
Solution Depth: 16
Path Cost: 94
Optimal: Yes
```

```
Algorithm: UCS
Average Time: 0.125089
Nodes Generated: 99412
Nodes Expanded: 97769
Maximum Frontier: 20422
Solution Depth: 16
Path Cost: 94
Optimal: Yes

Algorithm: IDDFS
Average Time: 0.077983
Nodes Generated: 199994
Nodes Expanded: 99994
Maximum Frontier: 17
Solution Depth: 16
Path Cost: 94
Optimal: Yes
```



TASK 10 – THEORY VS EXPERIMENT

The measured node counts should be interpreted against the graph structure. The binary-tree test has branching factor at most two and a goal at the final node ID, so BFS and IDDFS examine a large portion of the tree. IDDFS performs repeated visits to upper levels, which increases generated-node counts while keeping its maximum recursion frontier close to the solution depth. BFS stores a broad queue and parent map, so its frontier is much larger. UCS orders by accumulated edge cost; its explored-node count depends on the deterministic edge weights and may differ from BFS even though the route is unique. Small differences in elapsed time are caused by Python data structures, heap operations, recursion overhead, and machine load.

TASK 11 – FINAL DESIGN RECOMMENDATION

For the original medicine-delivery problem with different corridor travel times, UCS is the recommended route planner because it explicitly minimizes total travel-time cost and is complete with positive edge costs. BFS is simpler and can be faster when all moves have equal cost, but it is not cost-optimal with unequal edges. IDDFS is useful when memory is severely constrained and a shallow goal is expected, but it repeats expansions and does not optimize weighted cost. In a deployed hospital, UCS should be combined with dynamic edge-cost updates, obstacle sensing, and replanning. The experiment is a controlled graph model rather than a physical hospital benchmark; deployment decisions should be validated on realistic maps and hardware.

TASK 12 – DAY-4 DESIGN CHALLENGE: LOCALIZATION FAILURE

A. Impact analysis

Originally, the robot knew its exact start node. After a sensor fault, it knows only that it is at one of k possible junctions. The fully observable and known-start assumptions are therefore invalid. The robot's knowledge must be part of the state because two robots at the same physical node but with different possible-location sets require different plans.

B. Evaluation of original design

The original algorithms can still be used only after redefining a search state as a belief state: a set of possible physical locations. If the physical graph has V nodes, the number of possible belief states is at most 2^V (or 2^V−1 if the empty set is excluded). This is much larger than V, so direct blind search over all belief states is generally impractical.

C. Adaptation: belief-state search with local sensing

Modify the design to maintain a belief set B. After an action, update B by applying the transition model to every location in B. When a local sensor observation is available, filter B to locations consistent with that observation. Plan over belief states until every possible location has a safe route to the pharmacy docking station, or use a conditional policy that selects the next action based on the sensor observation. This is safer than pretending the robot has a known location. The worst-case belief-space size is exponential, but informative local sensing can shrink the set quickly.

D. Small illustrative experiment

The following experiment demonstrates the count of possible non-empty belief states, 2^k−1, and the effect of a perfect local sensor. Without sensing, a conservative plan must account for every possible starting junction. With a perfect sensor, the belief collapses to one location before route planning. The illustrative plan length is k steps without sensing (a conservative sequential verification policy) and 1 step of sensing plus the shortest route length of 1 in the simplified example. This is a conceptual demonstration, not a measured physical navigation result.

Python Program:

```
def display_belief_state(belief):
    print("Possible Robot Locations:", belief)
    print("Number of Possible Locations:", len(belief)
def without_sensor(k):
    print("\nWITHOUT SENSOR")
    belief = set(range(1, k + 1))
    print("Initial Belief State:")
    display_belief_state(belief)
    print("Localization Status: Uncertain")
    return belief
def with_sensor(k, actual_location):
    print("\nWITH SENSOR")
    belief = set(range(1, k + 1))

    print("Initial Belief State:")
    display_belief_state(belief)
    sensor_reading = actual_location
    belief = {sensor_reading}

    print("\nSensor Reading:", sensor_reading)

    print("Updated Belief State:")
    display_belief_state(belief)

    print("Localization Status: Confirmed")

    return belief
print("HOSPITAL ROBOT LOCALIZATION")
print("DAY 4: LOCALIZATION FAULT")

k_values = [2, 4, 8]

for k in k_values:

    print("\n================================")
    print("NUMBER OF POSSIBLE JUNCTIONS:", k)
    print("================================")
    actual_location = k
    belief_without = without_sensor(k)
    belief_with = with_sensor(k, actual_location)
    print("\nCOMPARISON")
    print("Possible Locations Without Sensor:",
          len(belief_without))
    print("Possible Locations With Sensor:",
          len(belief_with))
    print("Uncertainty Reduced:",
          len(belief without) - len(belief with))
```

Output:

```
HOSPITAL ROBOT LOCALIZATION
DAY 4: LOCALIZATION FAULT

================================
NUMBER OF POSSIBLE JUNCTIONS: 2
================================

WITHOUT SENSOR
Initial Belief State:
Possible Robot Locations: {1, 2}
Number of Possible Locations: 2
Localization Status: Uncertain

WITH SENSOR
Initial Belief State:
Possible Robot Locations: {1, 2}
Number of Possible Locations: 2

Sensor Reading: 2
Updated Belief State:
Possible Robot Locations: {2}
Number of Possible Locations: 1
Localization Status: Confirmed

COMPARISON
Possible Locations Without Sensor: 2
Possible Locations With Sensor: 1
Uncertainty Reduced: 1
```

CONCLUSION

This assignment formulated hospital medicine delivery as a graph-search problem and compared BFS, UCS, and IDDFS. Mathematical analysis shows that BFS and IDDFS have exponential tree-search growth in solution depth, while IDDFS uses much less memory. UCS adds priority-queue overhead but is appropriate when travel-time costs differ. The controlled experiments provide operation counts and performance measurements across increasing graph sizes. Under localization failure, the state must become a belief set; adding sensing and belief-state updates is the recommended modification.

REFERENCES

[1] Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach, 4th Edition, Pearson, 2021.

[2] Course assignment brief: Assignment 1 – CO1, Artificial Intelligence – CSA17 (provided source document).

[3] Python 3 documentation, collections.deque and heapq modules, https://docs.python.org/3/

States	Algorithm	Avg time (s)	Generated	Expanded	Max frontier	Depth	Path cost	Optimal?
1000	BFS	0.000499	1000	999	500	9	37	Yes
1000	UCS	0.000620	932	759	244	9	37	Yes
1000	IDDFS	0.000747	1994	994	10	9	37	Yes
10000	BFS	0.004092	10000	9999	5000	13	44	Yes
10000	UCS	0.006770	6380	4493	2129	13	44	Yes
10000	IDDFS	0.007926	19995	9995	14	13	44	Yes
100000	BFS	0.044753	100000	99999	50000	16	59	Yes
100000	UCS	0.081660	69764	52492	19701	16	59	Yes
100000	IDDFS	0.079100	199994	99994	17	16	59	Yes
1000000	BFS	0.416622	1000000	999999	500000	19	86	Yes
1000000	UCS	1.799026	984282	942347	178236	19	86	Yes
1000000	IDDFS	0.695487	1999993	999993	20	19	86	Yes

All three methods return the unique root-to-goal route in this tree, so the route cost is the same for a given seed and is optimal in this test graph. BFS and IDDFS do not generally guarantee cost optimality on arbitrary weighted graphs.